

ระบบลงทะเบียน & จัดการงาน

เทมเพลตเว็บแอปเริ่มต้นสำหรับมือใหม่ — สมัครสมาชิก / เข้าสู่ระบบ + ตัวอย่าง CRUD พร้อม deploy ขึ้น AWS ECS ได้จริงด้วยสคริปต์เดียว



React + Vite



Express (Node.js)



PostgreSQL + Prisma



Docker



AWS ECS Fargate

http://localhost:8080

เข้าสู่ระบบ

อีเมล

รหัสผ่าน

เข้าสู่ระบบ

ยังไม่มีบัญชี? [สมัครสมาชิก](#)

1. หน้าสมัครสมาชิก & เข้าสู่ระบบ

ผู้ใช้ใหม่สมัครด้วยอีเมล + รหัสผ่าน ระบบเข้ารหัสรหัสผ่านด้วย bcrypt แล้วออก JWT ให้ทันที

/register

สมัครสมาชิก

ชื่อ

สมชาย ใจดี

อีเมล

somchai@example.com

รหัสผ่าน (อย่างน้อย 6 ตัว)

.....

สมัครสมาชิก

มีบัญชีอยู่แล้ว? [เข้าสู่ระบบ](#)

/login

เข้าสู่ระบบ

อีเมล

somchai@example.com

รหัสผ่าน

.....

เข้าสู่ระบบ

ยังไม่มีบัญชี? [สมัครสมาชิก](#)

Login

รหัสผิวดตอบข้อความเดียว (ไม่บอกใบว่าอีเมลมีไหม) · เก็บ JWT ไว้ใน browser ใช้ต่อ 7 วัน

Register

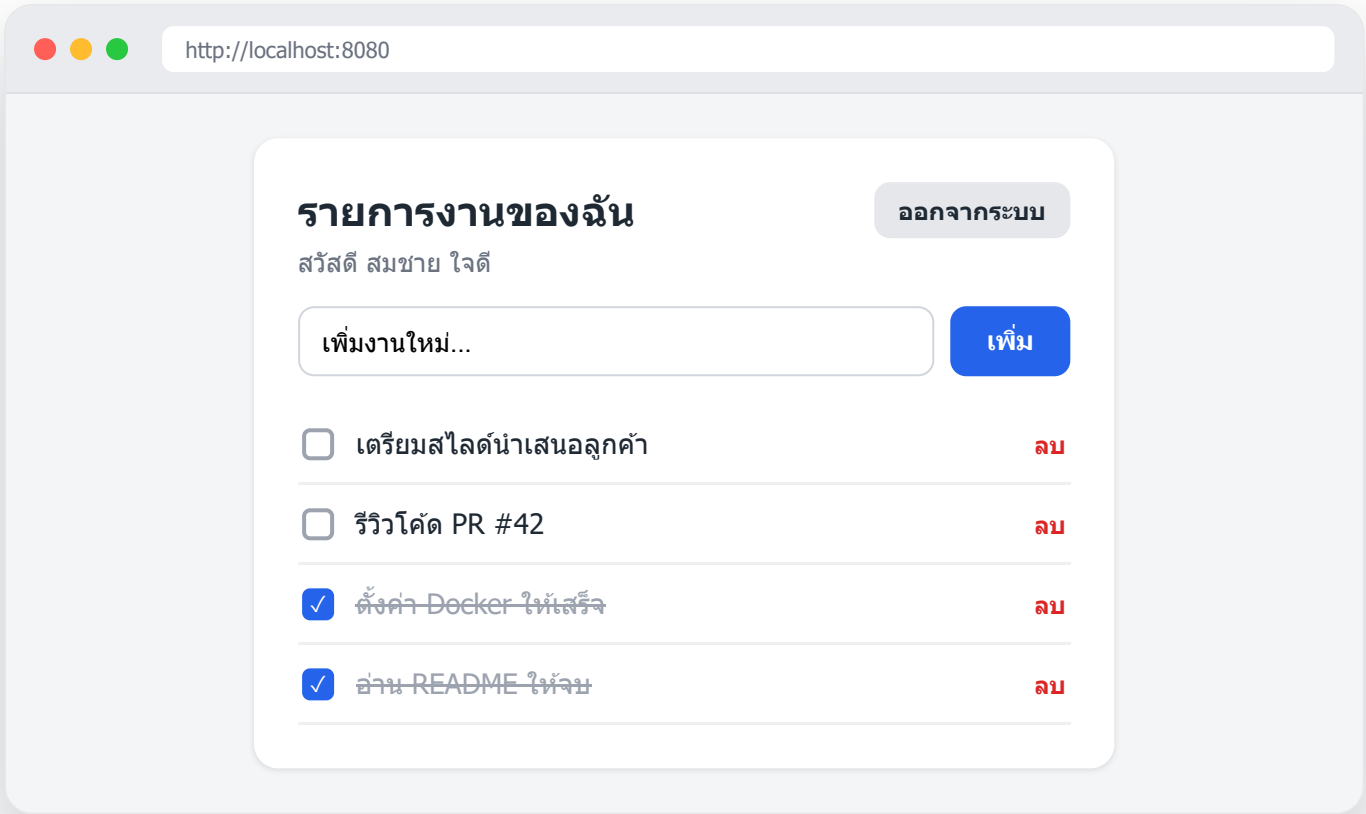
ตรวจอีเมลซ้ำ (ตอบ 409) · รหัสสลับกันไปเดือนทันที · สมัครเสร็จล็อกอินให้อัตโนมัติ

🔒 ความปลอดภัยที่ใส่มาให้

- รหัสผ่านไม่เคยถูกเก็บตรง ๆ — เก็บเป็น hash (bcrypt) เท่านั้น
- ทุก request ที่ต้องล็อกอิน จะแนบ `Authorization: Bearer <token>` และถูกตรวจสอบเสมอ
- ตรวจสอบข้อมูลขาเข้าด้วย zod ทุก endpoint

2. หน้ารายการงาน (ตัวอย่าง CRUD)

หลังล็อกอิน ผู้ใช้เห็นเฉพาะงานของตัวเอง เพิ่ม / ดีกว่าเสร็จ / ลบ ได้ครบวงจร



✔ **CRUD ครบ 4 อย่าง**

Create (เพิ่มงาน) · Read (ดูรายการ) · Update (ดีกเสร็จ/แก้ไข) · Delete (ลบ)

🔒 **แยกข้อมูลรายคน**

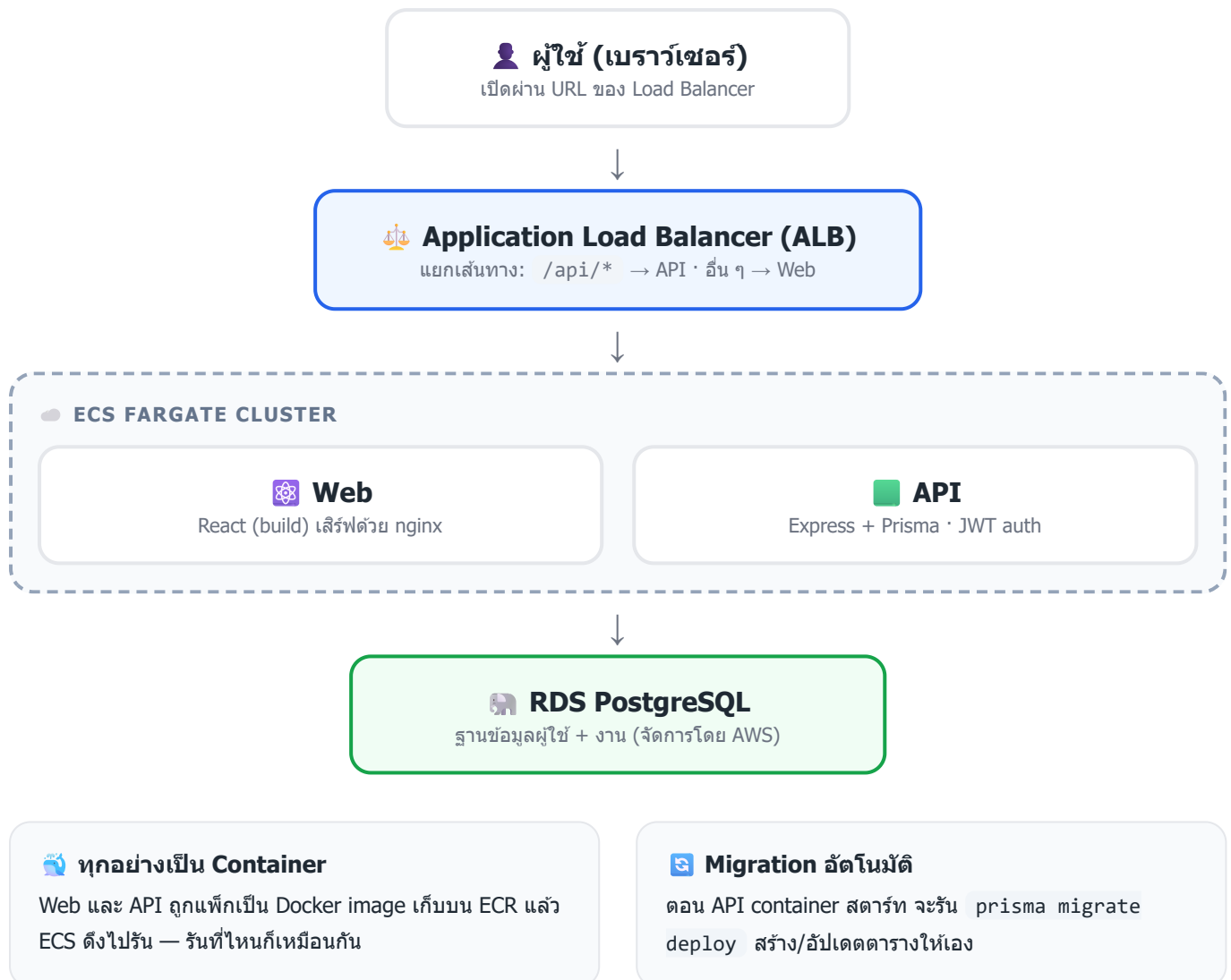
ทุก query กรองด้วย userId — ผู้ใช้เห็น/แก้/ลบได้เฉพาะงานของตัวเองเท่านั้น

API Endpoints

METHOD	PATH	หน้าที่	สิทธิ์
POST	/api/auth/register	สมัครสมาชิก	public
POST	/api/auth/login	เข้าสู่ระบบ	public
GET	/api/auth/me	ดูข้อมูลตัวเอง	auth
GET	/api/tasks	ดูงานทั้งหมดของฉัน	auth
POST	/api/tasks	เพิ่มงานใหม่	auth
PATCH	/api/tasks/:id	แก้ไข / ดีกเสร็จ	auth
DEL	/api/tasks/:id	ลบงาน	auth
GET	/health	เช็คสถานะ (ให้ AWS ใช้)	public

3. สถาปัตยกรรมระบบบน AWS

ทุกส่วนรันเป็น container บน ECS Fargate มี Load Balancer เป็นประตูหน้าคอยแยกเส้นทาง



4. ขั้นตอน Deploy ขึ้น AWS

รันทีเดียวจบด้วย `./deploy-all.sh` — สคริปต์สร้างทุกอย่างให้อัตโนมัติแบบ idempotent (รันซ้ำได้)

0

ตรวจสอบความพร้อม `00-check.sh`

เช็ค aws cli / docker และว่าล็อกอิน AWS แล้ว

1

Build & Push image `01-build-push.sh`

สร้าง Docker image ของ web + api แล้ว push ขึ้น ECR (คลัง image ของ AWS)

2

สร้างโครงสร้างพื้นฐาน `02-infra.sh`

VPC / Security Group / RDS PostgreSQL / ECS Cluster / Load Balancer + Target Group

3

Deploy ขึ้น ECS `03-deploy.sh`

สร้าง task definition + service ให้ container รันตลอด แล้วผูกกับ Load Balancer

```
# ครั้งแรก — สร้างทุกอย่าง
cp deploy/config.example.sh deploy/config.sh
# แก้ DB_PASSWORD และ JWT_SECRET ในไฟล์ config.sh
cd deploy && ./deploy-all.sh

# แก้โค้ดแล้ว deploy ใหม่
./01-build-push.sh && ./03-deploy.sh

# ลบทิ้งกันโดนคิดเงิน
./99-destroy.sh
```

⚠ ก่อนใช้งานจริงจัง ควรปรับเพิ่ม

ย้าย secret ไป AWS Secrets Manager · เพิ่ม HTTPS ที่ ALB · จำกัดสิทธิ์ IAM · ตั้ง desired-count > 1